```
  public int refunds()
  {
    return itsRefunds;
  }

  public void deposit()
  {
    itsCoins++;
  }

  public void refund()
  {
    itsRefunds++;
  }
}

class Locked extends Turnstile
{
  public void coin()
  {
    itsState = UNLOCKED;
    lock(false);
    alarm(false);
    deposit();
  }

  public void pass()
  {
    alarm(true);
  }
}


class Unlocked extends Turnstile
{
  public void coin()
  {
    refund();
  }

  public void pass()
  {
    lock(true);
    itsState = LOCKED;
  }
}
```

This example shows some of the useful features of the MONOSTATE pattern. It takes advantage of the ability for MONOSTATE derivatives to be polymorphic and the fact that MONOSTATE derivatives are themselves MONOSTATES. This example also shows how difficult it can sometimes be to turn a MONOSTATE into a normal class. The structure of this solution depends strongly on the MONOSTATE nature of `Turnstile`. If we needed to control more than one turnstile with this finite-state machine, the code would require some significant refactoring.

Perhaps you are concerned about the unconventional use of inheritance in this example. Having `Unlocked` and `Locked` derived from `Turnstile` seems a violation of normal OO principles. However, since `Turnstile` is

a MONOSTATE, there are no separate instances of it. Thus, `Unlocked` and `Locked` aren't really separate objects. Instead they are part of the `Turnstile` abstraction. `Unlocked` and `Locked` have access to the same variables and methods that `Turnstile` has access to.

Conclusion

It is often necessary to enforce the constraint that a particular object have only a single instantiation. This chapter has shown two very different techniques. SINGLETON makes use of private constructors, a static variable, and a static function to control and limit instantiation. MONOSTATE simply makes all variables of the object static.

SINGLETON is best used when you have an existing class that you want to constrain through derivation, and you don't mind that everyone will have to call the `instance()` method to gain access. MONOSTATE is best used when you want the singular nature of the class to be transparent to the users or when you want to employ polymorphic derivatives of the single object.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.
3. Ball, Steve, and John Crawford. Monostate Classes: The Power of One. Published in *More C++ Gems*, compiled by Robert C. Martin. Cambridge, UK: Cambridge University Press, 2000, p. 223.

17

NULL OBJECT



Faultily faultless, icily regular, splendidly null, Dead perfection, no more.

—Alfred Tennyson (1809–1892)

Consider the following code:

```
Employee e = DB.getEmployee("Bob");
if (e != null && e.isTimeToPay(today))
  e.pay();
```

We ask the database for an `Employee` object named "Bob." The `DB` object returns `null` if no such object exists. Otherwise, it returns the requested instance of `Employee`. If the employee exists, and if it is time to pay him, then we invoke the `pay` method.

We've all written code like this before. The idiom is common because the first expression of the `&&` is evaluated first in C-based languages, and the second is evaluated only if the first is `true`. Most of us have also been burned by forgetting to test for `null`. Common though the idiom may be, it is ugly and error prone.

We can alleviate the tendency toward error by having `DB.getEmployee` throw an exception instead of returning `null`. However, `try/catch` blocks can be even uglier than checking for `null`. Worse, the use of exceptions forces us to declare them in `throws` clauses. This makes it hard to retrofit exceptions into an existing application.

We can address these issues by using the NULL OBJECT pattern.[1] This pattern often eliminates the need to check for `null`, and it can help to simplify the code.

Figure 17-1 shows the structure. `Employee` becomes an interface that has two implementations. `EmployeeImplementation` is the normal implementation. It contains all the methods and variables that you would expect an `Employee` object to have. When `DB.getEmployee` finds an employee in the database, it returns an instance of `EmployeeImplementation`. `NullEmployee` is returned only if `DB.getEmployee` cannot find the employee.

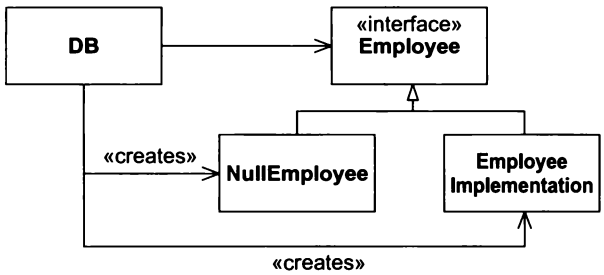


Figure 17-1 NULL OBJECT Pattern

`NullEmployee` implements all the methods of `Employee` to do "nothing." What "nothing" is depends on the method. For example, one would expect that `isTimeToPay` would be implemented to return `false`, since it is never time to pay a `NullEmployee`.

Using this pattern, we can change the original code to look like this:

```
Employee e = DB.getEmployee("Bob");
if (e.isTimeToPay(today))
  e.pay();
```

This is neither error prone nor ugly. There is a nice consistency to it. `DB.getEmployee` *always* returns an instance of `Employee`. That instance is guaranteed to behave appropriately, regardless of whether the employee was found or not.

Of course there will be many cases where we'll still want to know if `DB.getEmployee` failed to find an employee. This can be accomplished by creating a `static final` variable in `Employee` that holds the one and only instance of `NullEmployee`.

Listing 17-1 shows the test case for `NullEmployee`. In this case "Bob" does not exist in the database. Notice that the test case expects `isTimeToPay` to return `false`. Notice also that it expects the employee returned by `DB.getEmployee` to be `Employee.NULL`.

Listing 17-1

TestEmployee.java (Partial)

```
public void testNull() throws Exception
  {
    Employee e = DB.getEmployee("Bob");
```

1. [PLOPD3], p. 5. This delightful article, by Bobby Woolf, is full of wit, irony and practical advice.

```
    if (e.isTimeToPay(new Date()))
      fail();
    assertEquals(Employee.NULL, e);
  }
```

The DB class is shown in Listing 17-2. Notice that, for the purposes of our test, the `getEmployee` method just returns `Employee.NULL`.

Listing 17-2

DB.java

```
public class DB
{
  public static Employee getEmployee(String name)
  {
    return Employee.NULL;
  }
}
```

The `Employee` interface is shown in Listing 17-3. Notice that it has a `static` variable named `NULL` that holds an anonymous implementation of `Employee`. This anonymous implementation is the sole instance of the null employee. It implements `isTimeToPay` to return `false` and `pay` to do nothing.

Listing 17-3

Employee.java

```
import java.util.Date;
public interface Employee
{
  public boolean isTimeToPay(Date payDate);

  public void pay();

  public static final Employee NULL = new Employee()
  {
    public boolean isTimeToPay(Date payDate)
    {
      return false;
    }

    public void pay()
    {
    }
  };
}
```

Making the null employee an anonymous inner class is a way to make sure that there is only a single instance of it. There is no `NullEmployee` class per se. Nobody else can create other instances of the null employee. This is a good thing because we want to be able to say things like

```
if (e == Employee.NULL)
```

This would be unreliable if it were possible to create many instances of the null employee.

Conclusion

Those of us who have been using C-based languages for a long time have grown accustomed to functions that return `null` or `0` on some kind of failure. We presume that the return value from such functions needs to be tested. The NULL OBJECT pattern changes this. By using this pattern, we can ensure that functions always return valid objects, even when they fail. Those objects that represent failure do "nothing."

Bibliography

1. Martin, Robert, Dirk Riehle, and Frank Buschmann. *Pattern Languages of Program Design 3*. Reading, MA: Addison–Wesley, 1998.

18

The Payroll Case Study: Iteration One Begins



"Everything which is in any way beautiful is beautiful in itself, and terminates in itself, not having praise as part of itself."

—Marcus Aurelius, circa A.D. 170

Introduction

The following case study describes the first iteration in the development of a simple batch payroll system. You will find the user stories in this case study to be simplistic. For example, taxes are simply not mentioned. This is typical of an early iteration. It will provide only a very small part of the business value the customers need.

In this chapter we will do the kind of quick analysis and design session that often takes place at the start of a normal iteration. The customer has selected the stories for the iteration, and now we have to figure out how we are going to implement them. Such design sessions are short and cursory, just like this chapter. The UML diagrams you see here are no more than hasty sketches on a whiteboard. The real design work will take place in the next chapter, when we work through the unit tests and implementations.

Specification

The following are some notes we took while conversing with our customer about the stories that were selected for the first iteration:

- Some employees work by the hour. They are paid an hourly rate that is one of the fields in their employee record. They submit daily time cards that record the date and the number of hours worked. If they work more than 8 hours per day, they are paid 1.5 times their normal rate for those extra hours. They are paid every Friday.
- Some employees are paid a flat salary. They are paid on the last working day of the month. Their monthly salary is one of the fields in their employee record.
- Some of the salaried employees are also paid a commission based on their sales. They submit sales receipts that record the date and the amount of the sale. Their commission rate is a field in their employee record. They are paid every other Friday.
- Employees can select their method of payment. They may have their paychecks mailed to the postal address of their choice; they may have their paychecks held for pickup by the paymaster; or they can request that their paychecks be directly deposited into the bank account of their choice.
- Some employees belong to the union. Their employee record has a field for the weekly dues rate. Their dues must be deducted from their pay. Also, the union may assess service charges against individual union members from time to time. These service charges are submitted by the union on a weekly basis and must be deducted from the appropriate employee's next pay amount.
- The payroll application will run once each working day and pay the appropriate employees on that day. The system will be told to what date the employees are to be paid, so it will generate payments for records from the last time the employee was paid up to the specified date.

We could begin by generating the database schema. Clearly this problem could use some kind of relational database, and the requirements give us a very good idea of what the tables and fields might be. It would be easy to design a workable schema and then start building some queries. However, this approach will generate an application for which the database is the central concern.

Databases are *implementation details*! Considering the database should be deferred as long as possible. Far too many applications are inextricably tied to their databases because they were designed with the database in mind from the beginning. Remember the definition of abstraction: *the amplification of the essential and the elimination of the irrelevant*. The database is irrelevant at this stage of the project; it is merely a technique used for storing and accessing data, nothing more.

Analysis by Use Cases

Instead of starting with the data of the system, let's start by considering the behavior of the system. After all, it is the system's behavior that we are being paid to create.

One way to capture and analyze the behavior of a system is to create *use cases*. Use cases, as originally described by Jacobson, are very similar to the notion of user stories in XP. A use case is like a user story that has been elaborated with a little more detail. Such elaboration is appropriate once the user story has been selected for implementation in the current iteration.

When we perform use case analysis, we look to the user stories and acceptance tests to find out the kinds of stimuli that the users of this system provide. Then we try to figure out how the system responds to those stimuli.

For example, here are the user stories that our customer has chosen for the next iteration:

1. Add a new employee
2. Delete an employee
3. Post a time card
4. Post a sales receipt
5. Post a union service charge
6. Change employee details (e.g., hourly rate, dues rate.)
7. Run the payroll for today

Let's convert each of these user stories into an elaborated use case. We don't need to go into too much detail—just enough to help us think through the design of the code that fulfills each story.

Adding Employees

Use Case 1
Add New Employee

A new employee is added by the receipt of an `AddEmp` transaction. This transaction contains the employee's name, address, and assigned employee number. The transaction has three forms:

```
AddEmp <EmpID> "<name>" "<address>" H <hourly-rate>
AddEmp <EmpID> "<name>" "<address>" S <monthly-salary>
AddEmp <EmpID> "<name>" "<address>" C <monthly-salary> <commission-rate>
```

The employee record is created with its fields assigned appropriately.

Alternative 1:
An error in the transaction structure

If the transaction structure is inappropriate, it is printed out in an error message, and no action is taken.

Use case 1 hints at an abstraction. There are three forms of the `AddEmp` transaction, yet all three forms share the `<EmpID>`, `<name>`, and `<address>` fields. We can use the COMMAND pattern to create an `AddEmployeeTransaction` abstract base class with three derivatives: `AddHourlyEmployeeTransaction`, `AddSalariedEmployeeTransaction` and `AddCommissionedEmployeeTransaction`. (See Figure 18-1.)

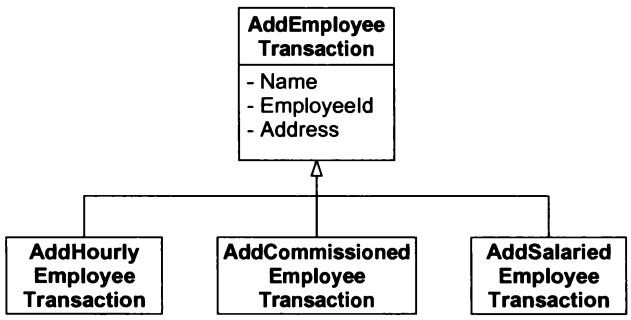


Figure 18-1 `AddEmployeeTransaction` Class Hierarchy

This structure conforms nicely to the Single-Responsibility Principle (SRP) by splitting each job into its own class. The alternative would be to put all these jobs into a single module. While this might reduce the number of classes in the system, and therefore make the system simpler, it would also concentrate all the transaction processing code in one place, creating a large and potentially error-prone module.

Use case 1 specifically talks about an employee record, which implies some sort of database. Again our predisposition to databases may tempt us into thinking about record layouts or the field structure in a relational database table, but we should resist these urges. What the use case is really asking us to do is to create an employee. What is the object model of an employee? A better question might be, What do the three different transactions create? In my view, they create three different kinds of employee objects, mimicking the three different kinds of `AddEmp` transactions. Figure 18-2 shows a possible structure.

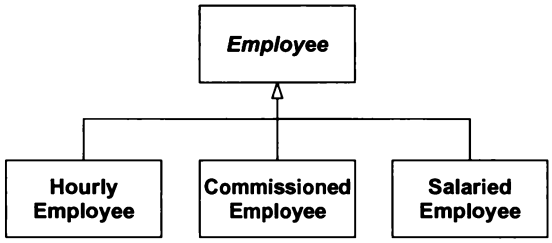


Figure 18-2 Possible `Employee` Class Hierarchy

Deleting Employees

Use Case 2
Deleting an Employee

Employees are deleted when a `DelEmp` transaction is received. The form of this transaction is as follows:

```
DelEmp <EmpID>
```

When this transaction is received, the appropriate employee record is deleted.

Alternative 1:
Invalid or unknown `EmpID`

If the `<EmpID>` field is not structured correctly, or if it does not refer to a valid employee record, then the transaction is printed with an error message, and no other action is taken.

This use case doesn't give me any design insights at this time, so let's look at the next.

Posting Time Cards

Use Case 3
Post a `Time Card`

On receiving a `TimeCard` transaction, the system will create a time-card record and associate it with the appropriate employee record.

```
TimeCard <Empld> <date> <hours>
```

Alternative 1:
The selected employee is not hourly

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

This use case points out that some transactions apply only to certain kinds of employees, strengthening the idea that the different kinds should be represented by different classes. In this case, there is also an association implied between time cards and hourly employees. Figure 18-3 shows a possible static model for this association.

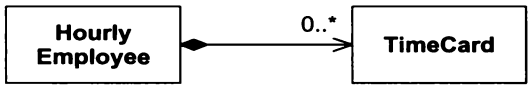


Figure 18-3 Association between `HourlyEmployee` and `TimeCard`

Posting Sales Receipts

Use Case 4
Posting a `Sales Receipt`

Upon receiving the `SalesReceipt` transaction, the system will create a new sales-receipt record and associate it with the appropriate commissioned employee.

```
SalesReceipt <EmpID> <date> <amount>
```

Alternative 1:
The selected employee is not commissioned

The system will print an appropriate error message and take no further action.

Alternative 2:
An error in the transaction structure

The system will print an appropriate error message and take no further action.

This use case is very similar to use case 3. It implies the structure shown in Figure 18-4.

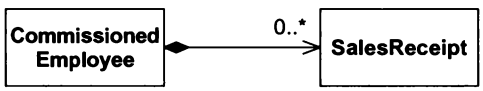


Figure 18-4 Commissioned Employees and Sales Receipts

Posting a Union Service Charge

Use Case 5
Posting a Union Service Charge

Upon receiving this transaction, the system will create a service-charge record and associate it with the appropriate union member.

```
ServiceCharge <memberID> <amount>
```

Alternative 1:
Poorly formed transaction

If the transaction is not well formed or if the `<memberID>` does not refer to an existing union member, then the transaction is printed with an appropriate error message.

This use case shows that union members are not accessed through employee IDs. The union maintains its own identification numbering scheme for union members. Thus, the system must be able to associate union

members and employees. There are many different ways to provide this kind of association, so to avoid being arbitrary, let's defer this decision until later. Perhaps constraints from other parts of the system will force our hand one way or another.

One thing is certain. There is a direct association between union members and their service charges. Figure 18-5 shows a possible static model for this association.

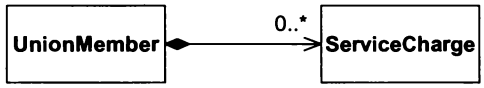


Figure 18-5 Union Members and Service Charges

Changing Employee Details

Use Case 6
Changing Employee Details

Upon receiving this transaction, the system will alter one of the details of the appropriate employee record. There are several possible variations to this transaction.

`ChgEmp <EmpID> Name <name>`	Change Employee Name
`ChgEmp <EmpID> Address <address>`	Change Employee Address
`ChgEmp <EmpID> Hourly <hourlyRate>`	Change to Hourly
`ChgEmp <EmpID> Salaried <salary>`	Change to Salaried
`ChgEmp <EmpID> Commissioned <salary> <rate>`	Change to Commissioned
`ChgEmp <EmpID> Hold`	Hold Paycheck
`ChgEmp <EmpID> Direct <bank> <account>`	Direct Deposit
`ChgEmp <EmpID> Mail <address>`	Mail Paycheck
`ChgEmp <EmpID> Member <memberID> Dues <rate>`	Put Employee in Union
`ChgEmp <EmpID> NoMember`	Remove Employee from Union

Alternative 1:
Transaction Errors

If the structure of the transaction is improper, or `<EmpID>` does not refer to a real employee, or `<memberID>` already refers to a member, then print a suitable error and take no further action.

This use case is very revealing. It has told us all the aspects of an employee that must be changeable. The fact that we can change an employee from hourly to salaried means that the diagram in Figure 18-2 is certainly invalid. Instead, it would probably be more appropriate to use the STRATEGY pattern for calculating pay. The `Employee` class could hold a strategy class named `PaymentClassification`, as in Figure 18-6. This is an advantage because we can change the `PaymentClassification` object without changing any other part of the `Employee` object. When an hourly employee is changed to a salaried employee, the `HourlyClassification` of the corresponding `Employee` object is replaced with a `SalariedClassification` object.

`PaymentClassification` objects come in three varieties. The `HourlyClassification` objects maintain the hourly rate and a list of `TimeCard` objects. The `SalariedClassification` objects maintain the monthly salary figure. The `CommissionedClassification` objects maintain a monthly salary, a commission rate, and a list of `SalesReceipt` objects. I have used composition relationships in these cases because I believe that `TimeCards` and `SalesReceipts` should be destroyed when the employee is destroyed.

The method of payment must also be changeable. Figure 18-6 implements this idea by using the STRATEGY pattern and deriving three different kinds of `PaymentMethod` classes. If an `Employee` object contains a

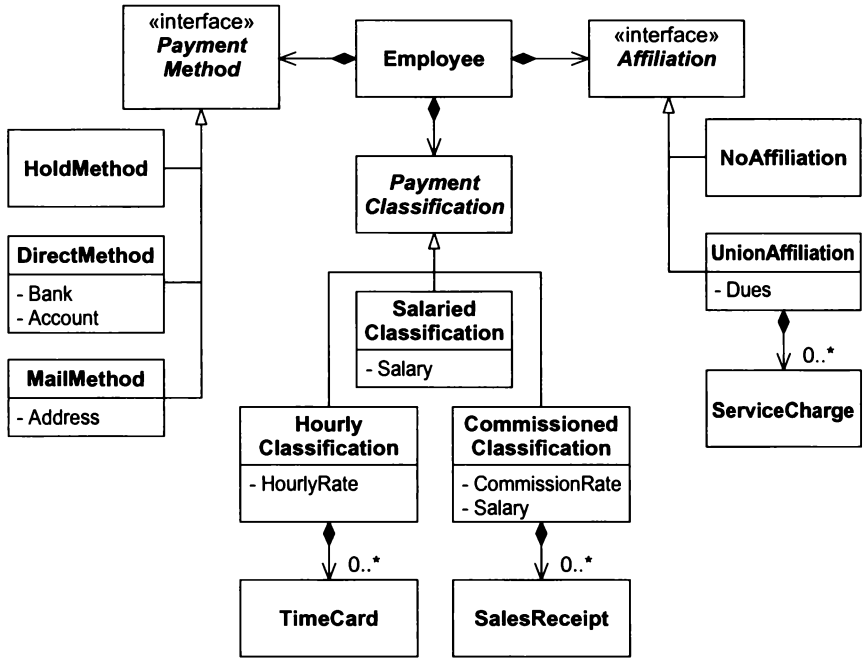


Figure 18-6 Revised Class Diagram for `Payroll -- The Core Model`

`MailMethod` object, the corresponding employee will have his paychecks mailed to him. The address to which the checks are mailed is recorded in the `MailMethod` object. If the `Employee` object contains a `DirectMethod` object, then his pay will be directly deposited into the bank account that is recorded in the `DirectMethod` object. If the `Employee` contains a `HoldMethod` object, his paychecks will be sent to the paymaster to be held for pickup.

Finally, Figure 18-6 applies the NULL OBJECT pattern to union membership. Each `Employee` object contains an `Affiliation` object, which has two forms. If the `Employee` contains a `NoAffiliation` object, then his pay is not adjusted by any organization other than the employer. However, if the `Employee` object contains a `UnionAffiliation` object, that employee must pay the dues and service charges that are recorded in that `UnionAffiliation` object.

This use of these patterns makes this system conform well to the Open-Closed Principle (OCP). The `Employee` class is closed against changes in payment method, payment classification, and union affiliation. New methods, classifications, and affiliations can be added to the system without affecting `Employee`.

Figure 18-6 is becoming our *core model* or architecture. It's at the heart of everything that the payroll system does. There will be many other classes and designs in the payroll application, but they will all be secondary to this fundamental structure. Of course, this structure is not cast in stone: It will be evolving along with everything else.

Payday

Use Case 7
Run the Payroll for Today

> Upon receiving the Payday transaction, the system finds all those employees that should be paid on the specified date. The system then determines how much they are owed and pays them according to their selected payment method.
>
> `Payday <date>`

Although it is easy to understand the intent of this use case, it is not so simple to determine what impact it has on the static structure of Figure 18-6. We need to answer several questions.

First, how does the `Employee` object know how to calculate its pay? Certainly if the employee is hourly, the system must tally up his time cards and multiply by the hourly rate. If the employee is commissioned, the system must tally up his sales receipts, multiply by the commission rate, and add the base salary. But where does this get done? The ideal place seems to be in the `PaymentClassification` derivatives. These objects maintain the records needed to calculate pay, so they should probably have the methods for determining pay. Figure 18-7 shows a collaboration diagram that describes how this might work.

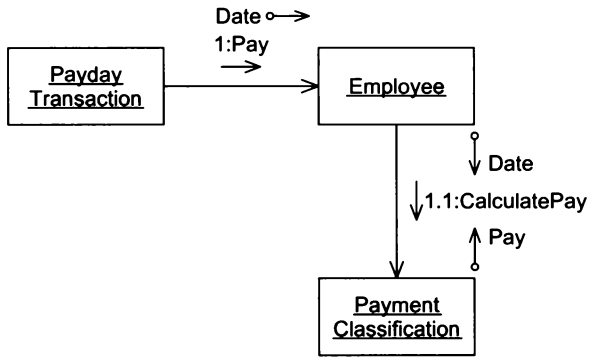


Figure 18-7 Calculating an Employee's Pay

When the `Employee` object is asked to calculate pay, it refers this request to its `PaymentClassification` object. The actual algorithm employed depends on the type of `PaymentClassification` that the `Employee` object contains. Figures 18-8 through 18-10 show the three possible scenarios.

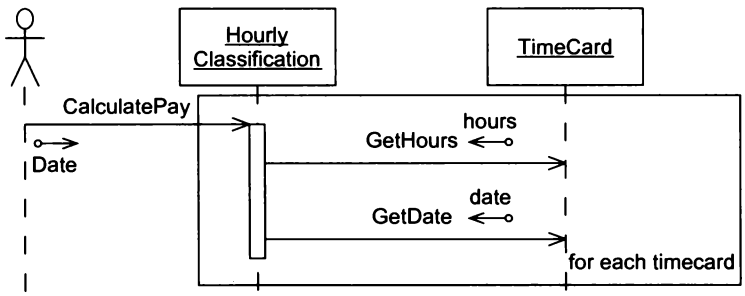


Figure 18-8 Calculating an Hourly Employee's Pay

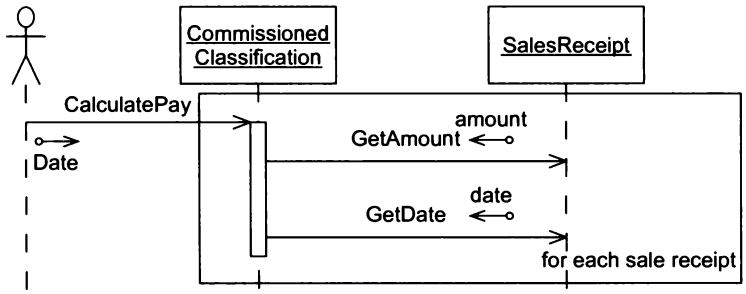


Figure 18-9 Calculating a Commissioned Employee's Pay

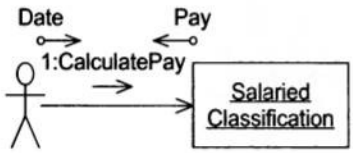
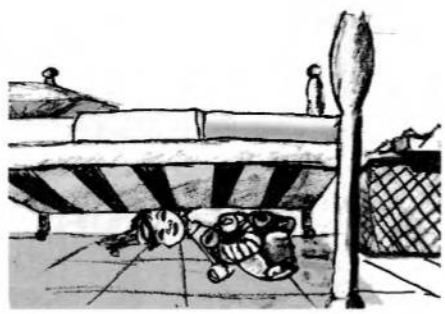


Figure 18-10 Calculating a Salaried Employee's Pay

Reflection: What Have We Learned?

We have learned that a simple use case analysis can provide a wealth of information and insights into the design of a system. Figures 18-6 through 18-10 came about by thinking about the use cases, that is, thinking about behavior.

Finding the Underlying Abstractions



To use the OCP effectively, we must hunt for abstractions and find those that underlie the application. Often these abstractions are not stated or even alluded to by the requirements of the application, or even the use cases. Requirements and use cases may be too steeped in details to express the generalities of the underlying abstractions.

What are the underlying abstractions of the Payroll application? Let's look again at the requirements. We see statements such as "Some employees work by the hour," "Some employees are paid a flat salary," and "Some [...] employees are paid a commission." This hints at the following generalization: "All employees are paid, but they are paid by different schemes." The abstraction here is that "All employees are paid." Our model of the `PaymentClassification` in Figures 18-7 through 18-10 expresses this abstraction nicely. Thus, this abstraction has already been found among our user stories by doing a very simple use-case analysis.

The Schedule Abstraction

Looking for other abstractions, we find "They are paid every Friday," "They are paid on the last working day of the month," and "They are paid every other Friday." This leads us to another generality: "All employees are paid according to some schedule." The abstraction here is the notion of the *schedule*. It should be possible to ask an `Employee` object whether a certain date is its payday. The use cases barely mention this. The requirements associate an employee's schedule with his payment classification. Specifically, hourly employees are paid weekly, salaried employees are paid monthly, and employees receiving commissions are paid biweekly; however, is this association essential? Might not the policy change one day so that employees could select a particular schedule or so that employees belonging to different departments or different divisions could have different schedules? Might not the schedule policy change independently of the payment policy? Certainly, this seems likely.

If, as the requirements imply, we delegated the issue of schedule to the `PaymentClassification` class, then our class could not be closed against issues of change in schedule. When we changed payment policy, we would also have to test schedule. When we changed schedules, we would also have to test payment policy. Both the OCP and the SRP would be violated.

An association between schedule and payment policy could lead to bugs in which a change to a particular payment policy caused incorrect scheduling of certain employees. Bugs like this may make sense to programmers, but they strike fear in the hearts of managers and users. They fear, and rightly so, that if schedules can be broken by a change to payment policy, then *any* change made *anywhere* might cause problems in *any* other unrelated part of the system. They fear that they cannot predict the effects of a change. When effects cannot be predicted, confidence is lost and the program assumes the status of "dangerous and unstable" in the minds of its managers and users.

Despite the essential nature of the schedule abstraction, our use-case analysis failed to give us any direct clues about its existence. To spot it required careful consideration of the requirements and an insight into the wiles of the user community. Overreliance on tools and procedures, and underreliance on intelligence and experience are recipes for disaster.

Figures 18-11 and 18-12 show the static and dynamic models for the schedule abstraction. As you can see, we've employed the STRATEGY pattern yet again. The `Employee` class contains the abstract `PaymentSchedule` class. There are three varieties of `PaymentSchedule` that correspond to the three known schedules by which employees are paid.

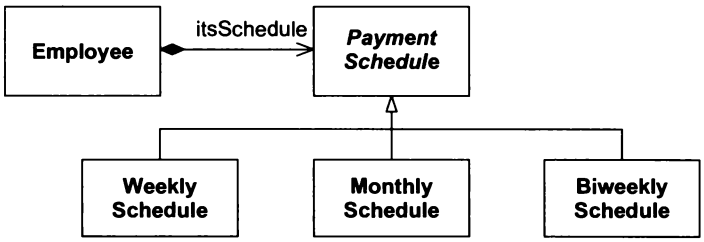


Figure 18-11 Static Model of a `Schedule` Abstraction

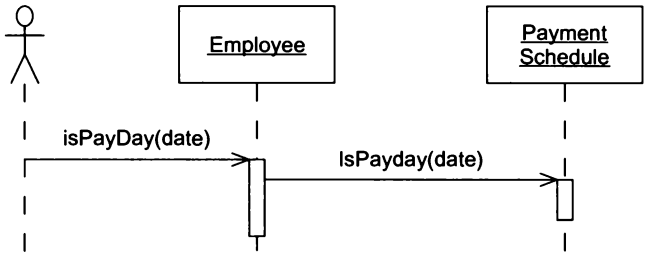


Figure 18-12 Dynamic Model of Schedule Abstraction

Payment Methods

Another generalization that we can make from the requirements is "All employees receive their pay by some method." The abstraction is the `PaymentMethod` class. Interestingly enough, this abstraction is already expressed in Figure 18-6.

Affiliations

The requirements imply that employees may have affiliations with a union; however, the union may not be the only organization that has a claim to some of an employee's pay. Employees might want to make automatic contributions to certain charities or have their dues to professional associations paid automatically. The generalization therefore becomes "The employee may be affiliated with many organizations that should be automatically paid from the employee's paycheck."

The corresponding abstraction is the `Affiliation` class that is shown in Figure 18-6. That figure, however, does not show the `Employee` containing more than one `Affiliation`, and it shows the presence of a `NoAffiliation` class. This design does not quite fit the abstraction we now think we need. Figures 18-13 and 18-14 show the static and dynamic models that represent the `Affiliation` abstraction.

The list of `Affiliation` objects has obviated the need to use the NULL OBJECT pattern for unaffiliated employees. Now, if the employee has no affiliation, his or her list of affiliations will simply be empty.

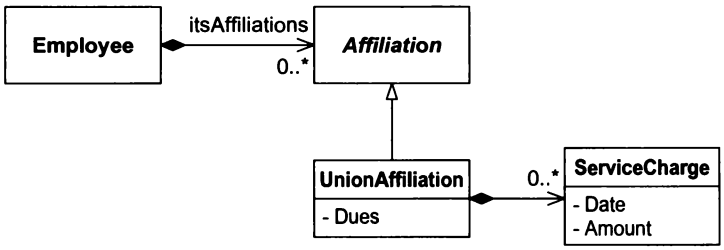


Figure 18-13 Static Structure of `Affiliation` Abstraction

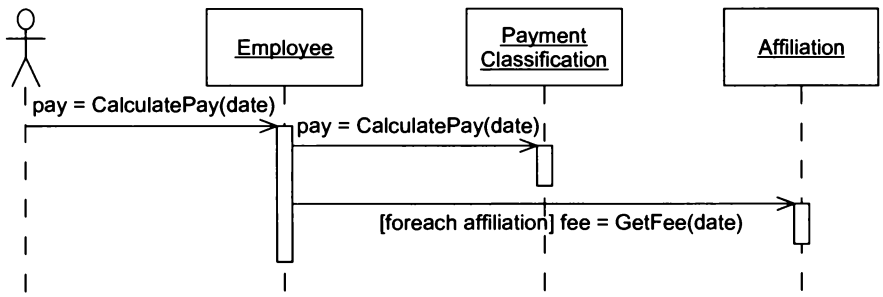


Figure 18-14 Dynamic Structure of `Affiliation` Abstraction

Conclusion

At the beginning of an iteration it is not uncommon to see the team assemble in front of a whiteboard and reason together about the design for the user stories that were selected for that iteration. Such a *quick design session* typically lasts less than an hour. The resulting UML diagrams, if any, may be left on the whiteboard, or erased. They are usually not commited to paper. The purpose of the session is to *start* the thinking process, and give the developers a common mental model to work from. The goal is *not* to nail down the design.

This chapter has been the textual equivalent to such a quick design Session.

Bibliography

1. Jacobson, Ivar. *Object-Oriented Software Engineering, A Use-Case-Driven Approach*. Wokingham, England: Addison–Wesley, 1992.

19

The Payroll Case Study: Implementation



It's long past time we started writing the code that supports and verifies the designs we've been spinning. I'll be creating that code in very small incremental steps, but I'll show it to you only at convenient points in the text. Don't let the fact that you only see fully formed snapshots of code mislead you into thinking that I wrote it in that form. In fact, between each batch of code you see, there will have been dozens of edits, compiles and test cases, each one making a tiny evolutionary change in the code.

You'll also see quite a bit of UML. Think of this UML as a quick diagram that I sketch on a whiteboard to show you, my pair partner, what I have in mind. UML makes a convenient medium for you and me to communicate by.

Figure 19-1 shows that we represent transactions as an abstract base class named `Transaction`, which has an instance method named `Execute()`. This is, of course, the COMMAND pattern. The implementation of the `Transaction` class is shown in Listing 19-1.

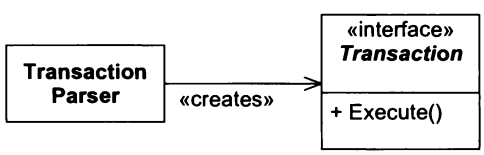


Figure 19-1 Transaction Interface

Listing 19-1

Transaction.h

```
#ifndef TRANSACTION_H
#define TRANSACTION_H

class Transaction
{
 public:
  virtual ~Transaction();
  virtual void Execute() = 0;
};

#endif
```

Adding Employees

Figure 19-2 shows a potential structure for the transactions that add employees. Note that it is within these transactions that the employees' payment schedule is associated with their payment classification. This is appropriate, since the transactions are contrivances instead of part of the core model. Thus, the core model is unaware of the association; the association is merely part of one of the contrivances and can be changed at any time. For example, we could easily add a transaction that allows us to change employee schedules.

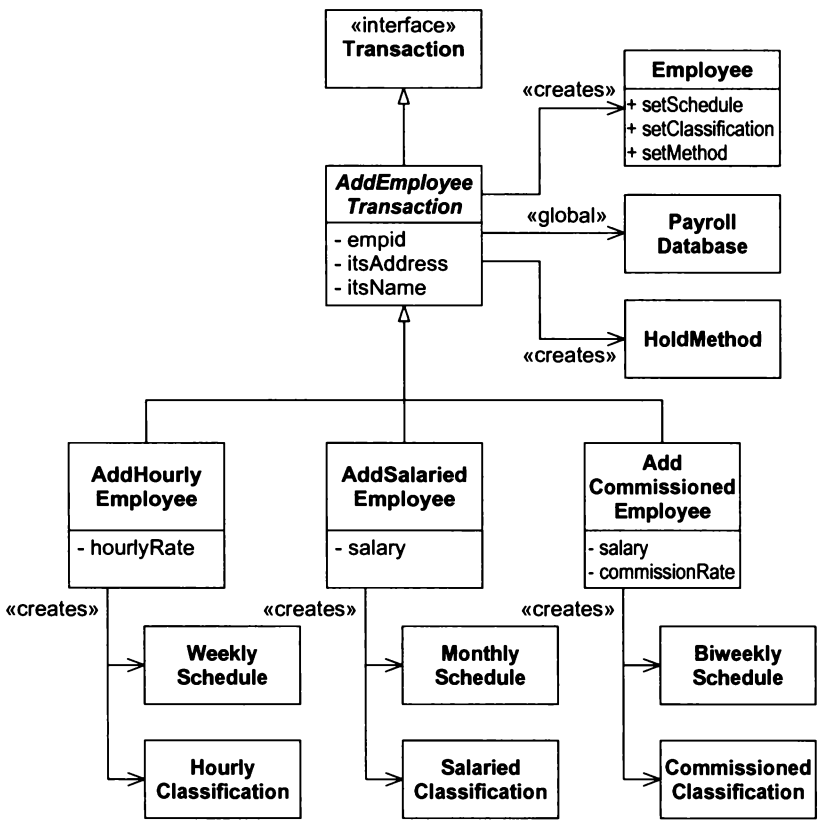


Figure 19-2 Static Model of `AddEmployeeTransaction`

Note, too, that the default payment method is to hold the paycheck with the paymaster. If an employee wants a different payment method, the change must be made with the appropriate `ChgEmp` transaction.

As usual, we begin writing code by writing tests first. Listing 19-2 is a test case that shows that the `AddSalariedTransaction` is working correctly. The code to follow will make that test case pass.

Listing 19-2

`PayrollTest::TestAddSalariedEmployee`

```
void PayrollTest::TestAddSalariedEmployee()
{
  int empId = 1;
  AddSalariedEmployee t(empId, "Bob", "Home", 1000.00);
  t.Execute();

  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert("Bob" == e->GetName());

  PaymentClassification* pc = e->GetClassification();
  SalariedClassification* sc = dynamic_cast<SalariedClassification*>(pc);
  assert(sc);

  assertEquals(1000.00, sc->GetSalary(), .001);
  PaymentSchedule* ps = e->GetSchedule();
  MonthlySchedule* ms = dynamic_cast<MonthlySchedule*>(ps);
  assert(ms);
  PaymentMethod* pm = e->GetMethod();
  HoldMethod* hm = dynamic_cast<HoldMethod*>(pm);
  assert(hm);
}
```

The Payroll Database

The `AddEmployeeTransaction` class uses a class called `PayrollDatabase`. This class maintains all the existing `Employee` objects in a `Dictionary` that are keyed by `empID`. It also maintains a `Dictionary` that maps union `memberIDs` to `empIDs`. The structure for this class appears in Figure 19-3. `PayrollDatabase` is an example of the FACADE pattern (page 173).

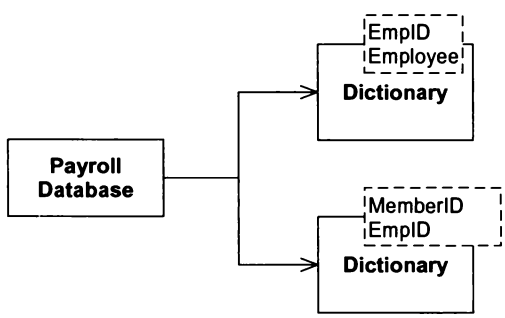


Figure 19-3 Static Structure of `PayrollDatabase`

Listings 19-3 and 19-4 show a rudimentary implementation of the `PayrollDatabase`. This implementation is meant to help us with our initial test cases. It does not yet contain the dictionary that maps member IDs to `Employee` instances.

Listing 19-3

PayrollDatabase.h

```
#ifndef PAYROLLDATABASE_H
#define PAYROLLDATABASE_H

#include <map>

class Employee;

class PayrollDatabase
{
 public:
  virtual ~PayrollDatabase();
  Employee* GetEmployee(int empId);
  void AddEmployee(int empid, Employee*);
  void clear() {itsEmployees.clear();}
 private:
  map<int, Employee*> itsEmployees;
};

#endif
```

Listing 19-4

PayrollDatabase.cpp

```
#include "PayrollDatabase.h"
#include "Employee.h"

PayrollDatabase GpayrollDatabase;

PayrollDatabase::~PayrollDatabase()
{
}

Employee* PayrollDatabase::GetEmployee(int empid)
{
  return itsEmployees[empid];
}

void PayrollDatabase::AddEmployee(int empid, Employee* e)
{
  itsEmployees[empid] = e;
}
```

In general, I consider database implementations to be details. Decisions about those details should be deferred as long as possible. Whether this particular database will be implemented with an RDBMS, flat files, or an OODBMS is irrelevant at this point. Right now, I'm just interested in creating the API that will provide database services to the rest of the application. I'll find appropriate implementations for the database later.

Deferring details about the database is an uncommon, but very rewarding, practice. Database decisions can usually wait until we have much more knowledge about the software and its needs. By waiting, we avoid the problem of putting too much infrastructure into the database. Rather, we implement just enough database facility for the needs of the application.

Using TEMPLATE METHOD to Add Employees

Figure 19-4 shows the dynamic model for adding an employee. Note that the `AddEmployeeTransaction` object sends messages to *itself* in order to get the appropriate `PaymentClassification` and `PaymentSchedule` objects. These messages are implemented in the derivatives of the `AddEmployeeTransaction` class. This is an application of the TEMPLATE METHOD pattern.

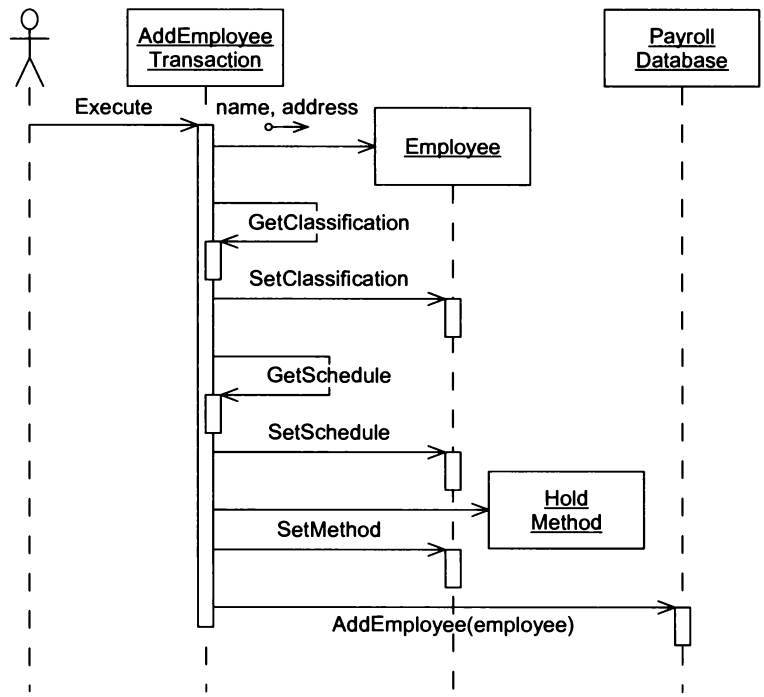


Figure 19-4 Dynamic Model for Adding an Employee

Listings 19-5 and 19-6 show the implementation of the TEMPLATE METHOD pattern in the `AddEmployeeTransaction` class. This class implements the `Execute()` method to call two pure virtual functions that will be implemented by derivatives. These functions, `GetSchedule()` and `GetClassification()`, return the `PaymentSchedule` and `PaymentClassification` objects that the newly created `Employee` needs. The `Execute()` method then binds these objects to the `Employee` and saves the `Employee` in the `PayrollDatabase`.

Listing 19-5

AddEmployeeTransaction.h

```
#ifndef ADDEMPLOYEETRANSACTION_H
#define ADDEMPLOYEETRANSACTION_H

#include "Transaction.h"
#include <string>

class PaymentClassification;
class PaymentSchedule;
```

```
class AddEmployeeTransaction : public Transaction
{
 public:
  virtual ~AddEmployeeTransaction();
  AddEmployeeTransaction(int empid, string name, string address);
  virtual PaymentClassification* GetClassification() const = 0;
  virtual PaymentSchedule* GetSchedule() const = 0;
  virtual void Execute();

 private:
  int itsEmpid;
  string itsName;
  string itsAddress;
};
#endif
```

Listing 19-6

AddEmployeeTransaction.cpp

```
#include "AddEmployeeTransaction.h"
#include "HoldMethod.h"
#include "Employee.h"
#include "PayrollDatabase.h"

class PaymentMethod;
class PaymentSchedule;
class PaymentClassification;

extern PayrollDatabase GpayrollDatabase;

AddEmployeeTransaction::~AddEmployeeTransaction()
{
}

AddEmployeeTransaction::
AddEmployeeTransaction(int empid, string name, string address)
  : itsEmpid(empid)
  , itsName(name)
  , itsAddress(address)
{
}

void AddEmployeeTransaction::Execute()
{
  PaymentClassification* pc = GetClassification();
  PaymentSchedule* ps = GetSchedule();
  PaymentMethod* pm = new HoldMethod();
  Employee* e = new Employee(itsEmpid, itsName, itsAddress);
  e->SetClassification(pc);
  e->SetSchedule(ps);
  e->SetMethod(pm);
  GpayrollDatabase.AddEmployee(itsEmpid, e);
}
```

Listings 19-7 and 19-8 show the implementation of the `AddSalariedEmployee` class. This class derives from `AddEmployeeTransaction` and implements the `GetSchedule()` and `GetClassification()` methods to pass back the appropriate objects to `AddEmployeeTransaction::Execute()`.

Listing 19-7

AddSalariedEmployee.h

```
#ifndef ADDSALARIEDEMPLOYEE_H
#define ADDSALARIEDEMPLOYEE_H

#include "AddEmployeeTransaction.h"

class AddSalariedEmployee : public AddEmployeeTransaction
{
 public:
  virtual ~AddSalariedEmployee();
  AddSalariedEmployee(int empid, string name,
                      string address, double salary);
  PaymentClassification* GetClassification() const;
  PaymentSchedule* GetSchedule() const;

 private:
  double itsSalary;
};
#endif
```

Listing 19-8

AddSalariedEmployee.cpp

```
#include "AddSalariedEmployee.h"
#include "SalariedClassification.h"
#include "MonthlySchedule.h"

AddSalariedEmployee::~AddSalariedEmployee()
{
}

AddSalariedEmployee::
AddSalariedEmployee(int empid, string name,
                    string address, double salary)
  : AddEmployeeTransaction(empid, name, address)
  , itsSalary(salary)
{
}


PaymentClassification*
AddSalariedEmployee::GetClassification() const
{
  return new SalariedClassification(itsSalary);
}

PaymentSchedule* AddSalariedEmployee::GetSchedule() const
{
  return new MonthlySchedule();
}
```

I leave the `AddHourlyEmployee` and `AddCommissionedEmployee` as exercises for the reader. Remember to write your test cases first.

Deleting Employees

Figures 19-5 and 19-6 present the static and dynamic models for the transactions that delete employees.

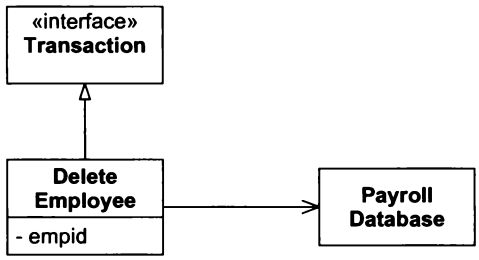


Figure 19-5 Static Model for `DeleteEmployee` Transaction

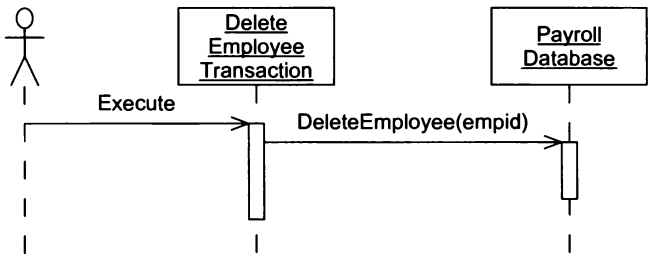


Figure 19-6 Dynamic Model for `DeleteEmployee` Transaction

Listing 19-9 shows the test case for deleting an employee. Listings 19-10 and 19-11 show the implementation of `DeleteEmployeeTransaction`. This is a very typical implementation of the COMMAND pattern. The constructor stores the data that the `Execute()` method eventually operates upon.

Listing 19-9

`PayrollTest::TestDeleteEmployee()`

```
void PayrollTest::TestDeleteEmployee()
{
  cerr << "TestDeleteEmployee" << endl;
  int empId = 3;
  AddCommissionedEmployee t(empId, "Lance", "Home", 2500, 3.2);
  t.Execute();
  {
    Employee* e = GpayrollDatabase.GetEmployee(empId);
    assert(e);
  }
  DeleteEmployeeTransaction dt(empId);
  dt.Execute();
  {
    Employee* e = GpayrollDatabase.GetEmployee(empId);
    assert(e == 0);
  }
}
```

Listing 19-10

DeleteEmployeeTransaction.h

```
#ifndef DELETEEMPLOYEETRANSACTION_H
#define DELETEEMPLOYEETRANSACTION_H

#include "Transaction.h"

class DeleteEmployeeTransaction : public Transaction
{
 public:
  virtual ~DeleteEmployeeTransaction();
  DeleteEmployeeTransaction(int empid);
  virtual void Execute();
 private:
  int itsEmpid;
};
#endif
```

Listing 19-11

DeleteEmployeeTransaction.cpp

```
#include "DeleteEmployeeTransaction.h"
#include "PayrollDatabase.h"

extern PayrollDatabase GpayrollDatabase;
DeleteEmployeeTransaction::~DeleteEmployeeTransaction()
{
}

DeleteEmployeeTransaction::DeleteEmployeeTransaction(int empid)
  : itsEmpid(empid)
{
}

void DeleteEmployeeTransaction::Execute()
{
  GpayrollDatabase.DeleteEmployee(itsEmpid);
}
```

Global Variables

By now you have noticed the `GpayrollDatabase` global. For decades, textbooks and teachers have been discouraging the use of global variables with good reason. Still, global variables are not intrinsically evil or harmful. This particular situation is an ideal choice for a global variable. There will only ever be one instance of the `PayrollDatabase` class, and it needs to be known by a very wide audience.

You might think that this could be better accomplished by using the SINGLETON or MONOSTATE patterns. It is true that these would serve the purpose. However, they do so by using global variables themselves. A SINGLETON or MONOSTATE is, by definition, a global entity. In this case I felt that a SINGLETON or MONOSTATE would smell of Needless Complexity. It's easier to simply keep the database instance in a global.

Time Cards, Sales Receipts, and Service Charges

Figure 19-7 shows the static structure for the transaction that posts time cards to employees. Figure 19-8 shows the dynamic model. The basic idea is that the transaction gets the `Employee` object from the `PayrollDatabase`, asks the `Employee` for its `PaymentClassification` object, and then creates and adds a `TimeCard` object to that `PaymentClassification`.

Notice that we cannot add `TimeCard` objects to general `PaymentClassification` objects; we can only add them to `HourlyClassification` objects. This implies that we must downcast the `PaymentClassification` object received from the `Employee` object to an `HourlyClassification` object. This is a good use for the `dynamic_cast` operator in C++, as shown later in Listing 19-15.

Listing 19-12 shows one of the test cases that verifies that time cards can be added to hourly employees. This test code simply creates an hourly employee and adds it to the database. Then it creates a `TimeCardTransaction` and invokes `Execute()`. Then it checks the employee to see if the `HourlyClassification` contains the appropriate `TimeCard`.

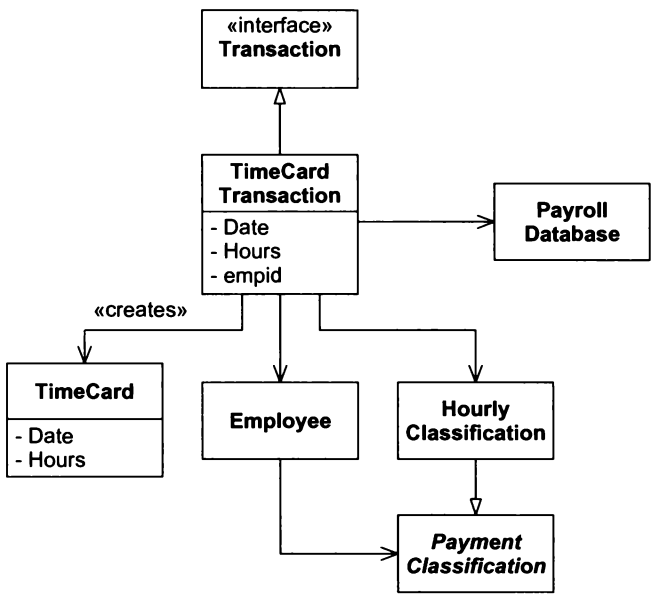


Figure 19-7 Static Structure of `TimeCardTransaction`

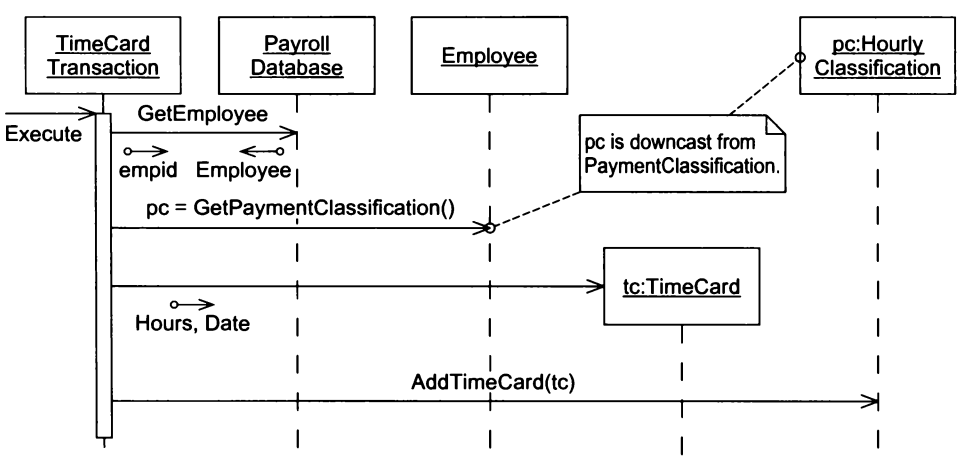


Figure 19-8 Dynamic Model for Posting a `TimeCard`

Listing 19-12

PayrollTest::TestTimeCardTransaction()

```
void PayrollTest::TestTimeCardTransaction()
{
  cerr << "TestTimeCardTransaction" << endl;
  int empId = 2;
  AddHourlyEmployee t(empId, "Bill", "Home", 15.25);
  t.Execute();
  TimeCardTransaction tct(20011031, 8.0, empId);
  tct.Execute();
  Employee* e = GpayrollDatabase.GetEmployee(empId);
  assert(e);
  PaymentClassification* pc = e->GetClassification();
  HourlyClassification* hc =
    dynamic_cast<HourlyClassification*>(pc);
  assert(hc);
  TimeCard* tc = hc->GetTimeCard(20011031);
  assert(tc);
  assertEquals(8.0, tc->GetHours());
}
```

Listing 19-13 shows the implementation of the `TimeCard` class. There's not much to this class right now. It's just a data class. Notice that I am using a long integer to represent dates. I'm doing this because I don't have a convenient `Date` class. I'm probably going to need one pretty soon, but I don't need it now. I don't want to distract myself from the task at hand, which is to get the current test case working. Eventually I will write a test case that will require a true `Date` class. When that happens, I'll go back and retrofit it into `TimeCard`.

Listing 19-13

TimeCard.h

```
#ifndef TIMECARD_H
#define TIMECARD_H

class TimeCard
{
 public:
  virtual ~TimeCard();
  TimeCard(long date, double hours);
  long GetDate() {return itsDate;}
  double GetHours() {return itsHours;}
 private:
  long itsDate;
  double itsHours;
};
#endif
```

Listings 19-14 and 19-15 show the implementation of the `TimeCardTransaction` class. Note the use of simple string exceptions. This is not particularly good long-term practice, but it suffices this early in development. After we get some idea of what the exceptions really ought to be, we can come back and create meaningful exception classes. Note also that the `TimeCard` instance is created only when we are sure we aren't going to throw an exception, so the throwing of the exception can't leak memory. It's very easy to create code that leaks memory or resources when throwing exceptions, so be careful. [1]

1. And run, don't walk, to buy *Exceptional C++* and *More Exceptional C++*, by Herb Sutter. These two books will save you much anguish, wailing, and gnashing of teeth over exceptions in C++.